

Group Project Guideline

From representation to evaluation — an end-to-end ML project in finance.

Instructor: Liu Peng (liupeng@smu.edu.sg) | TA: Youyou Xu (youyou.xu.2025@pbs.smu.edu.sg)

Weighting	40% of final grade
Team size	3–5 students
Project guideline released	Week 5
Group consultation	Week 8 — or other slots upon request
Final presentations	Weeks 9–10
Deliverables	Slide deck + reproducible notebook + in-class presentation

1. What this project is about

QF624 covers a sequence of machine-learning methods — from quantitative trading strategies and modern portfolio management, through linear and logistic regression, to deep neural networks, generalization, ML-based risk management, and reinforcement learning for portfolio control. The recurring lens is the same: a finance question becomes a math object — a representation, a model f , a loss ℓ , an evaluation — and the math object becomes a decision.

Your group project is where this lens gets exercised end-to-end. Pick a real finance problem, build an ML solution, and defend every design choice — from how you frame the math object, to how you split the data, to which metric maps onto the financial cost of being wrong. The depth of the model matters less than the fit between question, representation, method, and decision.

2. Choose ONE problem family

Each family below is anchored to a specific course week so you can see exactly which lectures will give you the methodological foundation. Pick one — open-ended exploration is discouraged.

Problem family	Course anchor	A sample finance question	Suitable evaluation metrics
Quantitative trading	Week 2 — Quantitative Trading Strategies	Construct an ML-driven entry/exit rule on a single asset class, evaluated	Net-of-cost Sharpe, hit rate, turnover,

		under realistic transaction costs and walk-forward backtesting.	drawdown, capacity considerations
Modern portfolio management	Week 3 — Modern Portfolio Management	Allocate across a multi-asset universe under risk, return, and turnover constraints; stress-test the allocation across regimes.	Out-of-sample Sharpe, max drawdown, turnover, tracking error, tail-loss measures
Return forecasting	Week 4 — Linear Regression	Forecast next-day or next-month equity returns from price- and fundamental-based features; convert into a tradable signal.	Out-of-sample R^2 , directional accuracy, IC / rank-IC, Sharpe of the resulting strategy
Credit risk classification	Week 5 — Logistic Regression	Predict 90-day default probability for a portfolio of unsecured personal loans; build a scorecard a credit officer can act on.	AUC, KS statistic, expected loss given threshold, calibration
Deep learning in finance	Weeks 6–7 — DNNs · Generalization	Use a neural network for return prediction, volatility forecasting, or option pricing surface fitting; compare to a linear baseline and demonstrate it generalizes.	Out-of-sample loss vs. baseline, train / val / test gap, sensitivity to hyperparameters
Risk management with ML	Week 8 — Risk Management using ML	Estimate VaR, ES, or counterparty exposure with an ML model; benchmark against parametric or historical methods, especially in stressed regimes.	Backtest exception rate, Kupiec / Christoffersen tests, expected-shortfall accuracy
RL for portfolio control & execution	Week 9 — RL in Portfolio Optimization	Train an agent to rebalance a portfolio or slice a block trade under transaction costs; show	Cumulative reward, net-of-cost Sharpe vs. baseline, slippage, policy stability

		the policy improves on a non-learning baseline.	
Market regime discovery	Cross-cutting — Unsupervised	Cluster macro/market data to identify hidden regimes, then show the regimes are useful for a downstream allocation or risk decision.	Cluster stability, regime-conditional Sharpe or drawdown improvement, silhouette / BIC

Other topics are welcome provided the question is concrete, the data is obtainable, and the output supports a financial decision.

3. The pipeline we expect to see

Every financial ML system can be read as a five-stage pipeline. Your notebook and slides should make each stage visible — not buried in code, but called out as labelled sections.

Stage	What you do	What the deliverable should show
01 Raw data	Gather signals — prices, returns, transactions, filings, balance-sheet ratios — and document source, sampling frequency, time coverage, and licensing.	A data section that names sources, sample size, time coverage, and any known biases.
02 Representation	Choose the right mathematical object: tabular $X \in \mathbb{R}^{n \times p}$, time series (x_t) , embeddings, or graph $G=(V,E)$. Engineer features, build labels, and pin a clear train / validation / test split that respects time order.	An explicit split diagram or table. No data point used for evaluation may have been seen at training time.
03 Model f0	Fit a model family appropriate to the representation, tune hyperparameters on validation data, and compare against at least one simple baseline (linear / equal-weight / persistence).	Side-by-side comparison with the baseline, and a one-sentence justification for the chosen model class.
04 Decision	Translate model output into the financial action: approve / decline, trade / hold, allocate weight, flag for review, slice an order.	A clearly stated decision rule, including the threshold or weighting logic and what determined it.

05 Outcome	Evaluate on the held-out test set with a metric that reflects the cost structure of the decision; report what would change if the model were deployed.	At least two metrics — one statistical, one financial — with confidence ranges, regime breakdown, or stress tests.
-------------------	--	--

Discipline rule: the test set is touched once, at the end. Tuning, model selection, and feature engineering use only the training and validation splits.

The four-component recipe — keep it visible in your slides

Most ML problems share one structure: **(1) representation** of inputs and labels, **(2) a model family** f_θ , **(3) a loss** ℓ that defines what counts as a good prediction or decision, and **(4) an evaluation** that asks whether performance survives on unseen future data. State each one explicitly in your method slide.

4. Deliverables

① Slide deck	Maximum 15 slides. Tell a coherent story: problem framing → representation → model → decision → evaluation & limitations. The deck should open with the financial decision the work supports, not with the model. Submit one calendar day before your presentation.
② Reproducible notebook	A Jupyter notebook (or .py modules with a README) using the course Python stack — NumPy, pandas, matplotlib, scikit-learn — and any framework appropriate to your method (PyTorch, TensorFlow, stable-baselines3 for RL). The notebook must: • Run end-to-end with one command on a clean environment. • Pin library versions in a requirements file. • Show the train / validation / test split explicitly and respect time order. • Include a small representative data sample or a working data-source link.
③ In-class presentation	15–20 minutes during your assigned cohort week (9 or 10), followed by a 5-minute Q&A. Every team member must speak. Be ready to explain any line of code on request.

5. Assessment rubric

Three dimensions, equally weighted within the 40% project grade. Each dimension is scored against the concrete criteria below.

Dimension	What we look for
-----------	------------------

Problem solving	• The financial decision is named upfront and used to justify every choice that follows. • Representation, model, loss, and evaluation are each defended against a simple baseline. • Evaluation is honest about uncertainty, generalization, and regime risk.
Team work	• The deck reads as one coherent story rather than glued pieces. • Workload is visibly shared; every member can answer questions on the full pipeline. • Peer evaluation may be used to break ties.
Communication	• A non-specialist can follow the slides; jargon is defined or avoided. • Code is reproducible and well-commented. • Figures and tables are informative, not decorative.

6. Common ML failure modes — and how to avoid them

Four pitfalls account for the majority of broken financial ML systems. Your project should explicitly address each one — even briefly — to show you have thought about why a model might fail outside the dev environment.

Failure mode	Why it bites in finance	How to detect or mitigate it
Regime change	A model trained on a long bull market can fail catastrophically when volatility regime, correlation structure, or policy environment shifts (e.g., 2020 COVID shock, 2022 rate cycle).	Backtest across multiple regimes; report performance separately by regime; use walk-forward or rolling-window evaluation rather than a single split.
Lookahead bias / leakage	Training inadvertently uses information that would not have been available at prediction time — revised macro data, restated fundamentals, point-in-time index changes, normalization computed across the test period — inflating backtest results.	Use point-in-time data wherever possible; align every feature to its real-time release timestamp; fit any normalization or feature engineering on the training split only.
Overfitting to noise	With many candidate features and limited data, a model can memorize spurious patterns that vanish on new data — the standard	Apply regularization (Week 7 material); cap model complexity; keep the test set sealed until the very end; report the train / validation / test gap honestly.

	story behind a 99% backtest and a 50% live performance.	
Label quality	Inconsistent definitions — what counts as “default,” “fraud,” or a “winning trade” — yield different models. Bad labels cap how good any model can ever be.	Document the labelling rule precisely; sanity-check labels on a small hand-reviewed sample; report sensitivity of results to plausible alternative definitions.

7. Working norms

- **Decision first.** Open every artefact — slide, README, plot — by stating the financial decision the work supports.
- **Honest evaluation.** Report what didn’t work, not just what did. Walk-forward where time matters; never report a tuned-on-test number.
- **Reproducibility.** A teammate (or the instructor) should be able to run your notebook end-to-end without messaging you.
- **Originality and AI-tool use.** Cite all data sources and external code. Generative AI tools may be used as assistants, but the modelling choices, analysis, and writing must be the team’s own. Be ready to explain any line of code, derivation, or paragraph on request — including in the Q&A.

8. Where to get help

Instructor	Liu Peng · liupeng@smu.edu.sg · office hours posted on eLearn
Teaching assistant	Youyou Xu · youyou.xu.2025@pbs.smu.edu.sg · Telegram group for class discussion
Group consultation	Week 7 (3 Jun), or other slots upon request
Recommended reading	• The Statistics and Machine Learning with R Workshop • Bayesian Optimization: Theory and Practice Using Python • Quantitative Trading Strategies with Python • Quantitative Risk Management Using Python • Deep Learning Generalization <i>All available at SMU Library.</i>